后训练学习计划 • 第二周（9/14 - 9/20）

> 本周定位：**Phase 1 第一周——把「语言建模的损失长什么样」彻底搞清楚**。主题是 makemore（bigram → MLP 字符级语言模型）+ 概率与信息论（MLE/MAP、熵/交叉熵/KL、`p-y` 梯度）+ Adam 与学习率。上周你学会了「怎么算梯度」，这周学会「梯度要去优化的那个东西到底是什么」。
>
> 用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，`> 注` 块 = 注释（可跳过，不影响主线）。本周不发布博客（第一篇《我从零训了一个 GPT》在 Week 5），但每天产出的素材必须归档，Week 5 写作时才不会空手。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	概率唤醒：MLE → 交叉熵；`softmax+CE` 梯度 = `p-y`	手推稿 + `ce_check.py` 打印 `CE GRAD CHECK OK`	6h
Day 2 周二	熵 / 交叉熵 / KL；PPL = exp(CE) 与 torch 对拍	`kl_asym.py` + `ppl_check.py` 双 OK + 自己算出的 bigram 熵下限	6h
Day 3 周三	makemore bigram：查表版 + 梯度版	两个实现 loss 逼近你算的熵（查表 ≤1e-3、梯度版 ≤H+0.15）+ 采样出可读人名	6.5h
Day 4 周四	makemore MLP：one-hot 上下文 → 嵌入	`train_mlp.py` 的 loss 明显低于 bigram 版 + 窗口消融小表	6.5h
Day 5 周五	Adam / AdamW / 动量 / lr 调度	手写 AdamW 与 torch 版 loss 相对差 <1e-6 + `opt_sweep.png`（16 条曲线，含 1 条炸掉）	6.5h
Day 6 周六	收口：数据换莎士比亚 + repo 整理 + gaps 记账	`week02/makemore/` 可复现 + NOTES ≥15 条 + `week02-gaps.md` 映射表	6h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + Week 3 预习	验收 ≥6/7 项通过	4.5h

> 注：本周**卡时上限 0.5h、预估花费 ¥0**（纯本地 CPU，makemore 的模型只有几万个参数）。唯一允许的开窗时间是 Day 6 任务 2 的 30 分钟「CPU vs GPU 跑同一个 bigram」体验，超预算就是没忍住，去看 AutoDL 控制台计费明细。**Week 2-5 的总卡时上限是 30h**，本周省下的都要留给 Week 5 的 M1。
>
> 注：本周最大风险是**手推数学拖成无底洞**。砍单顺序（做不完就按这个砍）：Day 2 任务 5 JS 距离 → Day 4 任务 4 窗口消融 → Day 5 任务 4 的读图三句 → **其余不许砍**。不许砍的三项里，Day 1 的梯度对拍、Day 2 的 PPL 对拍、Day 5 的手写 AdamW 是这周真正的能力增量，也是 Week 5 M1 训练和 Week 6 SFT 的地基。
>
> 注：本周**故意不含** BatchNorm、Dropout、Embedding 的高级用法、分布式。makemore 视频里 Karpathy 后面还讲了 BatchNorm 和学习率手动搜索（那是 `bigger.py` 的内容），本周不追，Week 4 会用更系统的方式讲训练技巧。别提前跳到调包微调——那是 Week 6 的事。

二、开始前：环境、目录与上周欠账（约 30 分钟，前一晚做完）

1. 复用 Week 1 的虚拟环境（**不要在 conda base 里装 torch**，Windows 下 conda 的 MKL 会和 torch 自带的 OpenMP 打架，报 `OMP: Error #15: Access is violated` / `libiomp5md.dll already initialized`），并把目录建好（bash 里路径用正斜杠，反斜杠会被吞）：

```
cd E:/study/posttrain
source .venv/Scripts/activate        # PowerShell 用 .venv\Scripts\Activate.ps1
pip install "numpy>=1.24" matplotlib torch -i https://pypi.tuna.tsinghua.edu.cn/simple
python -c "import numpy,torch,matplotlib;print(numpy.__version__, torch.__version__)"   # 不许出现 OMP: Error #15
mkdir -p E:/study/posttrain/week02/{day1,day2,day3,day4,day5,day6,data,makemore}
mkdir -p E:/study/posttrain/llm-posttraining-lab/{notes,blogs/drafts}
```

2. 上周三样东西必须能打开：`E:/study/posttrain/day2/engine.py`、`day2/nn.py`、`day4/week1-gaps.md`（Day 5 要直接 import 前两个）。Git Bash 自带 `curl`，Day 3 下数据够用。
3. 如果 Week 1 有 `WANDB_MODE=offline` 攒下的离线目录没补传，今天顺手 `wandb sync <离线目录>`（约 5 分钟）；传不动就在 `week02-gaps.md` 再记一行「周末补」，**不要在今晚跟网络较劲**（学术加速对 wandb 常无效）。

完成判据：`python -c` 那行打印出三个版本号且**没有** `OMP: Error #15`；`week02/` 六个子目录存在；`day2/engine.py` 能被 `python -c "import sys;sys.path.insert(0,'E:/study/posttrain/day2');from engine import Value;print(Value(3)*Value(4))"` 打印 `Value(12)`。

> 注：最后那条 import 检查别嫌琐碎——Week 1 的 `Value` 是本周 Day 3/Day 5 的地基，路径错了你会在 Day 3 晚上 11 点才发现自己在跟 `ModuleNotFoundError` 搏斗。

Day 1 • 周一：概率唤醒——交叉熵不是玄学，是 MLE 换了件衣服

今日总览（6h）：今天把 Week 1 自测第 2、3 题的两个 B 类空洞正面闭合。晚上 11 点前你要拿到三样东西：① 一张手推稿，从「数据集似然」推到「最小化平均交叉熵」，再推到 `MAP = MLE + 权重衰减`；② `softmax+CE` 的梯度手推，写成 `p - onehot(y)`；③ 一个 numpy 对拍脚本，打印 `CE GRAD CHECK OK`。**这三样是后训练一切损失函数的祖先**——SFT 是 CE、RM 的 Bradley-Terry 是 BCE 家族、DPO 是 sigmoid 里的 CE 变体，Week 8 手推 DPO 时你今天的稿子会被反复翻出来。

任务 1：开工对账——从 `week1-gaps.md` 里清哪几条（约 20 分钟）

打开 `E:/study/posttrain/day4/week1-gaps.md`，本周 Day 1 认领下面这些条目（**逐条抄进 `E:/study/posttrain/week02/day1/today.md` 顶部**，做完当天在后面标 `[x] 动作`）：

1. **`softmax + 交叉熵、p-y 梯度`**（Week 1 自测第 2 题，B 类，Week 1 明确推给「Week 2 · makemore」）→ 今天任务 4 手推 + 任务 5 数值对拍闭合。

2. `KL 散度与熵的关系`（第 3 题，B 类，推给「Week 2 · 采样与分布匹配」）→ 明天 Day 2 任务 1/2 闭合，今天只做前置（任务 2 里 CE 的定义）。
3. `Adam / 偏差修正 / 学习率`（第 5 题，B 类，推给「Week 2 · 训练循环与调 lr」）→ Day 5 闭合。
4. **A 类遗留与杂账（有则必清，无则跳过）**：自测第 1 题手推梯度若未闭合 → 今天任务 4 顺带补；第 6 题 attention 前向 → Week 3 Day 1 重做；`train_moons` acc 停在 90-95%、被砍的 `moons.png`、gaps 里任何写了「Week 2」却没指定天的条目 → 全部登记到 `week02-gaps.md` 的「上周遗留」小节，Day 6/Day 7 结算。

完成判据：`week02/day1/today.md` 里 ≥5 行，其中 B 类三条（1/2/3）**必须**各写一行「闭合在哪天的哪个任务」；`week02-gaps.md` 已新建且有「上周遗留」小节。

> 注：为什么把「认领 gaps」排在今晚第一件事而不是 Day 7？因为这三条洞的补课动作分布在 Day 1/2/5，**你要先知道今天的推稿是为了周四的哪道题**，否则手推会退化成抄笔记。面试官最爱问的也不是「你知道 CE 吗」，而是「CE 的梯度为什么能化简成 p−y，实现时为什么要和 softmax 合并成一个算子」——这一问的答案就在今天的任务 4、5。

任务 2：MLE → 交叉熵，一张手推稿（约 70 分钟）

1. 拿纸和笔（不要打字，推稿要拍照进 `notes/`，Week 8 推 DPO 时你会感谢这个习惯）。
2. 按这条链推，每步标注你用了什么规则：
   - 数据集似然：语言模型把一段文本建模成 `P(w_1..w_T) = Π_t P(w_t | w_{<t})`（自回归分解）。
   - MLE：最大化 `Σ_t log P(w_t | w_{<t})`，等价于最小化它的相反数。
   - 逐位置真实分布是 one-hot（观测到的 token 概率为 1），于是 `-log P(w_t|·)` 正好是**该位置的交叉熵** `H(p_t, q_t) = -Σ_v p_t(v) log q_t(v)`。
   - 对数据集取平均 → **平均 MLE = 平均交叉熵 = 模型分布与数据分布之间的交叉熵**。
   - 再补一行：`H(p,q) = H(p) + D_KL(p‖q)`，数据分布的熵 `H(p)` 是常数，所以**最小化 CE 等价于最小化 KL**，这也回答了 Week 1 自测第 3 题的后半句「为什么训练时用 CE 而不是直接最小化 KL」。
3. 检查你能否不看稿口头讲 90 秒：「SFT 的 loss 为什么是 CE、为什么只对 assistant token 算」。

完成判据：推稿上出现五个式子且链条不断：自回归分解、对数似然、one-hot 交叉熵、数据集平均、`H = H(p) + KL`。手机拍照存 `week02/day1/derivation-mle-ce.jpg`。

> 注：`F.cross_entropy` 的入参是 **logits 而不是概率**（内部做了 log-softmax，这就是 Week 1 自测第 2 题「为什么要把 softmax 和 CE 合并成一个算子」的工程答案：分开写会先上溢再取 log，数值不稳且多一次归一化）。把 logits 当概率传进去**不会报错**，只会让 loss 变小、训练「看起来在收敛」——这是后训练岗位最难查的一类 bug，Day 2 任务 4 会用数值对拍把它钉死。

任务 3：MAP = MLE + 权重衰减（约 40 分钟）

1. 同一张纸接着推：给参数加高斯先验 `θ ~ N(0, σ²I)`，写出后验 `log P(θ|D) = log P(D|θ) − ‖θ‖²/(2σ²) + const`，说明**最大后验（MAP）= 负对数似然 + L2 惩罚**，也就是 `weight_decay` 的概率学含义。
2. 再写一行结论：AdamW 之所以不叫 Adam+L2，是因为 L2 项进了梯度会被 Adam 的二阶矩 `√v̂` 缩放，从而**在梯度大的地方把权重衰减削弱掉**；AdamW 把衰减从梯度里摘出来直接乘在参数上（Day 5 任务 1 会亲

手看到这个差别）。

3. 在 `NOTES.md` 写两条：「weight decay 是先验」「AdamW 的 W 是 decoupled」。

完成判据：`NOTES.md` 新增 ≥3 条且包含上面两句原话（自己话术可改，意思必须对）；推稿上 MAP 那一行有 `1/(2σ²)` 与 `weight_decay` 的对应关系。

任务 4：手推 `softmax + CE` 的梯度 = `p - y`（约 70 分钟）

1. 单样本：`L = -log p_k`，其中 `p_i = e^{z_i}/Σ_j e^{z_j}`，`k` 是正确类别。
2. 分两种情况求 `∂L/∂z_i`：`i = k` 与 `i ≠ k`，把 `∂p_k/∂z_i` 用商的求导写全（这一步 Week 1 自测时最容易跳，跳过就等于没懂）。
3. 合并成向量形式：`∂L/∂z = p - onehot(y)`。
4. batch 形式：`∂L/∂z = (p - onehot(y)) / N`，**并解释为什么必须有 1/N**（`reduction='mean'` 是除有效 token 数，不是除 numel；这个区别在 Week 6 的 SFT loss masking 和 packing 上会直接决定你的等效 lr）。
5. 口头预演面试官的两层追问：①「为什么梯度是 p−y 这么干净的形式，这说明了 CE 和 softmax 的什么关系」②「把 CE 换成 MSE 会怎样」（提示：MSE + softmax 的梯度里会多出 `p(1-p)` 因子，概率接近 0 或 1 时梯度消失，学得极慢）。

完成判据：手稿上有 `∂L/∂z_i = p_i - [i=k]` 这一行**且**推导过程包含 `∂p_k/∂z_i` 的两种情况；任务 5 的数值对拍通过（那才是真正判据）。

任务 5：numpy 实现 + 数值稳定 + 梯度对拍（约 110 分钟）

新建 `week02/day1/ce_check.py`。**先自己写实现，卡住再看下面的骨架**——直接抄等于今天没做。

```
"""交叉熵的两种写法 + 解析梯度 vs 数值梯度对拍。统一 float64。"""
import numpy as np

rng = np.random.default_rng(0)          # seed 写死:对拍结论必须可复现
B, V = 5, 7
z = rng.normal(size=(B, V)) * 3.0       # float64,默认就是
y = rng.integers(0, V, size=(B,))

def softmax_np(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def ce_np(z, y):                        # 输入是 logits,内部做稳定的 log-softmax
    z = z - z.max(axis=-1, keepdims=True)
    logp = z - np.log(np.exp(z).sum(axis=-1, keepdims=True))
    return -logp[np.arange(z.shape[0]), y].mean()

print("np.exp(1000.0) =", np.exp(1000.0), "| 稳定 softmax:", softmax_np(np.array([[1000., 1001., 999.]])))

p = softmax_np(z)
grad = (p - np.eye(V)[y]) / B            # 解析梯度:就是你手推的 p - onehot,除以 N
eps = 1e-6
num = np.zeros_like(z)
for i in range(B):                        # 中心差分:每个元素加减 eps 各跑一次前向
    for j in range(V):
        zp, zm = z.copy(), z.copy()
        zp[i, j] += eps
        zm[i, j] -= eps
        num[i, j] = (ce_np(zp, y) - ce_np(zm, y)) / (2 * eps)

diff = np.abs(grad - num).max()
print(f"max|analytic - numeric| = {diff:.2e}")
assert diff < 1e-7, f"CE 梯度对拍失败 diff={diff}"
print("CE GRAD CHECK OK")
```

完成判据：终端打印 `CE GRAD CHECK OK` 且 `diff` 在 **1e-9 ~ 1e-7 量级**；`np.exp(1000.0)` 那行打印出 `inf` 与 RuntimeWarning；把 `ce_np` 里的 `z - z.max(...)` 删掉再跑一次，把 `nan` 或 `inf` 的现象写进 NOTES（这就是「为什么合并算子」的第二次证据）。

> 注：中心差分步长 `eps=1e-6` 在 float64 下的误差量级约 `eps² + tol/eps ≈ 1e-12 + 1e-10`，所以 1e-7 的门槛是宽松可用的。**换成 float32 这个对拍会假失败**（diff 掉到 1e-3 量级），这不是梯度错，是浮点精度不够——本周所有数值对拍统一 float64，写进 NOTES 第一条。
>
> 注：如果你把 `np.eye(V)[y]` 写成 `np.eye(B)[y]`，或把 batch 维和词表维搞反，torch/numpy **会直接报 shape 错**，这反而是好事；真正坑的是维度刚好能广播错的写法（比如 `(p - y)` 里 y 忘了 one-hot），它会静默算出一个"能收敛但学不好"的梯度。数值梯度对拍是唯一的裁判，这个习惯要带到 Week 8 手写 DPO。

任务 6：字符级 vocab 与编解码 roundtrip（约 50 分钟）

字符级语言模型没有「词」，只有「字符表 + 两个约定」。新建 `week02/day1/vocab.py`，实现一个 `CharVocab`：构造时从文本收集 `chars = sorted(set(text))`，维护 `stoi: {字符: 下标}` 与 `itos: {下标: 字符}` 两个互逆字

典，提供 `encode(s) -> List[int]` 与 `decode(ids) -> str`，并在词表里留一个 `.` 作为「序列起止」符（Day 3 的约定是 `.` + 内容 + `.`）。

Day 3 会直接 import 这个类，所以接口不许自己改名。

完成判据：`python vocab.py` 里 `__main__` 的 roundtrip 断言通过并打印 `VOCAB ROUNDTRIP OK`；用 3 行注释写清「`stoi` 和 `itos` 各自拿什么当 key」——这两个字典搞混是 Day 3 采样出乱码的头号原因。

今日验收（6h）：`CE GRAD CHECK OK` + 推稿两张（MLE→CE、p−y）+ `VOCAB ROUNDTRIP OK` + NOTES 新增 ≥4 条。缺一项就地补，补不动就写进 `week02-gaps.md` 并留到 Day 7 的前 1 小时——**不要熬夜到 1 点**，周四周五的手感比今晚的第 4 项值钱。

> 注：删掉 `np.exp` 溢出那行不算完成。你会在 Day 3 训练 bigram 时看到 loss 从 `nan` 出发的惨剧，那时装死的人是你自己。

Day 2 • 周二：熵、交叉熵、KL——以及「你的模型最好能到什么 loss」

今日总览（6h）：今天闭合 Week 1 自测第 3 题（KL 与熵），并拿到一个本周最重要的**数字锚点**：你自己从数据算出来的 bigram 经验条件熵。它是任何 bigram 语言模型 loss 的不可突破下限，明天训练完就要拿这个数去比。晚上还要让 `ppl_check.py` 打印 `PPL CROSSCHECK OK`——用 numpy 实现对拍 torch 的 `F.cross_entropy`。

任务 1：熵 / 交叉熵 / KL 三件套手推（约 70 分钟）

1. 写四个定义（纸或 md 都行，但要自己写不抄）：
   - 熵 `H(p) = −Σ p log p`，单位取 nat（用自然对数；bit 与 nat 的换算是 `ln2`，面试爱问）。
   - 交叉熵 `H(p,q) = −Σ p log q`。
   - KL `D_KL(p‖q) = Σ p log(p/q) = H(p,q) − H(p)`。
   - 相对熵非负（Jensen 不等式），说明**为什么 `D_KL ≥ 0` 意味着「loss 有下限」**。
2. 口述三句话并写进 NOTES：① KL 为什么不对称（加权用的是 p 的分布，p 有质量而 q 没有 → 惩罚爆炸；反过来 p 无质量的地方 q 有质量 → 几乎不受罚）② 为什么训练用 CE 而不是 KL（差一个常数 H(p)）③ 为什么 forward KL 让模型"摊平"、reverse KL 让模型"抓一个峰"（后者是变分推断/RLHF 里 KL 惩罚用 `KL(π_θ‖π_ref)` 的原因，Week 9 会真撞上）。
3. 提前预告：Week 9 GRPO 的「KL 受控 = 健康」判据、DPO 的 β 控制「偏离 ref 的强度」，说的都是今天这个量。现在不求甚解，写一行「Week 8/9 会回来用它」。

完成判据：四个定义 + 三句话齐；NOTES 新增 ≥4 条。

> 注：`D_KL(p‖q)` 里 p 为 0、q 为正的项按 `0·log0 = 0` 处理，但 p 为正、q 为 0 的项是 `+inf`。这不是数学洁癖：Day 3 你用 bigram 查表模型给某个从未出现过的字符对赋概率 0，再拿它去算「以经验分布为 p」的 KL，就会打印出 `inf`——那一刻你就真的懂「为什么语言模型必须平滑（temperature/加一平滑/top-k）」而不是背一条规则。

任务 2：numpy 验证 KL 不对称（约 60 分钟）

新建 `week02/day2/kl_asym.py`：

```
import numpy as np
np.seterr(divide="ignore", invalid="ignore")      # 让 inf 打印出来而不是抛异常

def kl(p, q):
    """真 KL:p=0 的项按 0*log0 = 0 处理;q=0 且 p>0 的项就是 +inf,不要把它抹平。"""
    return float(np.sum(np.where(p > 0, p * np.log(p / q), 0.0)))

p = np.array([0.9, 0.1])
q = np.array([0.0, 1.0])                          # q 在 p 有 0.9 质量的地方是 0
print(f"KL(p||q) = {kl(p, q)}")                   # inf
print(f"KL(q||p) = {kl(q, p):.6f}")               # 2.302585 = ln 10,有限值
assert np.isinf(kl(p, q)) and abs(kl(q, p) - np.log(10)) < 1e-9

p3 = np.array([0.9, 0.1, 0.0])
q3 = np.array([0.1, 0.45, 0.45])                 # 两个都有限,依然不相等
print(f"KL(p3||q3) = {kl(p3, q3):.6f}   KL(q3||p3) = {kl(q3, p3):.6f}")
assert abs(kl(p3, q3) - kl(q3, p3)) > 1e-3, "KL 居然对称了,检查实现"
print("KL ASYMMETRY OK")
```

再加两个验证：① 自己造一个三项混合分布与它的"摊平"拟合，比较 forward / reverse KL 各自的 argmin（用 numpy 枚举 200 个候选二维分布即可，不用优化器）；② 交叉熵分解数值检查 `H(p,q) == H(p) + KL(p‖q)`（`np.allclose`）。

完成判据：打印 `KL ASYMMETRY OK`；`KL(p‖q)` 打印为 `inf` 而 `KL(q‖p)` 是 **2.302585**（= ln 10，脚本里已 assert）；两个三维有限值不相等；NOTES 记下这两个三维数各是多少，并写一句「哪个方向惩罚'漏掉'，哪个方向惩罚'摊太开'」。

任务 3：算出你的 bigram 经验条件熵 = loss 下限（约 70 分钟）

先把数据下下来（约 3.2 万个英文名字，Week 3 才换莎士比亚）：

```
cd E:/study/posttrain/week02/data
curl -L -o names.txt https://raw.githubusercontent.com/karpathy/makemore/master/names.txt
wc -l names.txt            # 记下真实行数,别用记忆里的数
```

raw 链接 404 就去 `github.com/karpathy/makemore` 仓库首页找 `names.txt`，或直接 `git clone` 整个仓库再拷出来。

然后新建 `week02/day3/bigram_entropy.py`（放 day3 是因为明天要用它，今天先算出来）：

```
"""从数据直接算 bigram 经验条件熵:任何 bigram LM 的平均 CE 都不可能低于它。"""
import numpy as np, math
path = "E:/study/posttrain/week02/data/names.txt"
names = [n.strip() for n in open(path) if n.strip()]
seqs = [["."] + list(n) + ["."] for n in names]
chars = sorted({c for s in seqs for c in s}) + ["."]
chars = sorted(set(chars))
V = len(chars); stoi = {c: i for i, c in enumerate(chars)}
pairs = [(stoi[a], stoi[b]) for s in seqs for a, b in zip(s[:-1], s[1:])]
N = len(pairs)
cnt = np.zeros((V, V), dtype=np.float64)
for a, b in pairs:
    cnt[a, b] += 1
prev = cnt.sum(axis=1)
cond = np.divide(cnt, prev[:, None], out=np.zeros_like(cnt), where=prev[:, None] > 0)
nz = cond > 0
H = -np.sum((prev / N)[:, None] * np.where(nz, np.log(np.where(nz, cond, 1.0)), 0.0))
print(f"V={V} N={N} names={len(names)}")
print(f"uniform 猜测的 CE      = {math.log(V):.6f}   (= ln V, 也就是随机基线)")
print(f"bigram 经验条件熵(下限) = {H:.6f}")
print(f"对应 PPL               = {math.exp(H):.4f}  (随机基线 PPL = {V})")
```

把输出的 `V`、`N`、`H` 三个数抄进 `NOTES.md` 顶部，标成「明天 loss 的下界与基线」。

完成判据：脚本跑通且 `V == 27`（26 字母 + `.`；不等就说明你的序列约定和 Week 3 的实现不一致，现在对齐比明天 debug 便宜）；`H` 落在 **2.0 ~ 2.5** 之间（`ln 27 ≈ 3.296` 是上界，随机猜就是这个数）；NOTES 里三个数齐全。

> 注：`V` 不等于 27 的常见原因有两个：漏了结束符 `.`（→26），或者把换行符/空行当字符（→28 以上）。这不是抠字眼：Day 3 的两个实现都要用同一个 `stoi`，否则「训练 loss 与你算的熵」根本不是同一个问题，比大小毫无意义。

任务 4：PPL = exp(CE)，与 torch 的 `F.cross_entropy` 对拍（约 90 分钟）

新建 `week02/day2/ppl_check.py`：

```
"""numpy 版 CE/PPL 对拍 torch F.cross_entropy。统一 float64,seed 写死。"""
import numpy as np
import torch
import torch.nn.functional as F

rng = np.random.default_rng(7)
B, T, V = 4, 6, 27
z = rng.normal(size=(B, T, V)) * 2.0
y = rng.integers(0, V, size=(B, T))

def ce_np(z, y):
    z = z - z.max(axis=-1, keepdims=True)
    logz = z - np.log(np.exp(z).sum(axis=-1, keepdims=True))
    B, T = y.shape
    # 三个索引一起给，才是"取第 (b,t) 位置上真实类别那一格"；只写两个会去索引 T 维
    pick = logz[np.arange(B)[:, None], np.arange(T)[None, :], y]
    return -pick.mean()

ce_mine = ce_np(z, y)
ce_torch = F.cross_entropy(torch.tensor(z).reshape(-1, V), torch.tensor(y).reshape(-1)).item()
ppl_mine = float(np.exp(ce_mine))
ppl_torch = float(np.exp(ce_torch))
print(f"CE  mine {ce_mine:.10f}  torch {ce_torch:.10f}  diff {abs(ce_mine-ce_torch):.2e}")
print(f"PPL mine {ppl_mine:.6f}  torch {ppl_torch:.6f}")
assert abs(ce_mine - ce_torch) < 1e-12, "CE 对拍失败,先查 axis 和 gather"
print("PPL CROSSCHECK OK")

# 第二个断言:均匀分布的 logits -> CE = ln V, PPL = V(27),即「随机猜测」基线
z_uni = np.zeros((B, T, V))
print("uniform CE =", ce_np(z_uni, y), "PPL =", np.exp(ce_np(z_uni, y)))
```

再补两问并把答案写进 NOTES：① `F.cross_entropy` 要求输入是 `(N, C)`（类别维在**第 1 维**），所以 3D 的 `(B,T,V)` 必须先 `reshape(-1, V)`、标签同步 `reshape(-1)`。动手验一下：把上面那行改成不 reshape 的写法，看它报什么错——你会看到 `IndexError: index 22 is out of bounds for axis 1 with size 6`，因为 torch 把 `T=6` 当成了类别数。**这类"形状约定"的错必须今天撞一次**，Week 6 做 loss masking、Week 8 做 DPO 时都是同一个坑 ② Week 5 训 GPT 时 eval loss 2.2 对应的 PPL 大概是多少，和均匀基线 27 比差几倍。

完成判据：打印 `PPL CROSSCHECK OK`（diff < 1e-12）；uniform 那行打印出 `CE ≈ ln 27 ≈ 3.296`、`PPL ≈ 27`，两个数抄进 NOTES——**以后任何语言建模实验，loss/PPL 都要和这个基线比，而不是和"感觉"比**。

> 注：torch 里 `ignore_index` 的**默认值就是 -100**，这正是 SFT 框架给 prompt 段填的那个数——所以框架里没有"手动跳过 prompt"的代码，它只是复用了这个默认参数。类别维永远是 `C` 那一维，GPT 出 `(B,T,V)` 时靠 reshape 而不是靠 transpose；`ignore_index=-1` 是分割任务里另一种习惯（-1 常被用作"背景/忽略"标签），别和 -100 混用。今天先记住形状约定；明天你手写 bigram 时如果按 `(V,B,T)` 传，torch 会静默地把 V 当 batch 算出一个奇怪的数——它不报错，只报错你的直觉。

任务 5：Jensen-Shannon 距离 + 分布可视化（约 45 分钟，可砍项）

1. `week02/day2/jsd.py`：用昨天的 `kl` 实现 `M = (P+Q)/2`、`JS = 0.5*KL(P‖M) + 0.5*KL(Q‖M)`，验证 JS 对称、非负、`JS(P,P) ≈ 0`（float64 <1e-15）三件事。

2. 用 matplotlib 画两分布对比图存 `jsd.png`（**必须** `matplotlib.use("Agg")` + `savefig`，别 `plt.show()`，Windows 无显示环境会挂住进程）。NOTES 写一句「为什么 RLHF/DPO 里说 KL 不说 JS」：要优化的量得能写成对数概率差，KL 能直接从 logits 估，JS 要两次前向。

完成判据：三个断言全过 + `week02/day2/jsd.png` 存在。砍掉不影响主线，砍了就记进 gaps。

任务 6：收工笔记（约 25 分钟）

NOTES 写：今天最意外的一个数值现象（大概率是 `KL(p‖q)` 打印成 `inf` 而 `KL(q‖p)` 只有 2.3026）+ 明天的三个锚点数（`V`、`N`、`H`）。

完成判据：NOTES.md 有当天两条新条目，且 `H`、`V`、`N` 三个数在笔记里能被第二天不开脚本就查到。

今日验收（6h）：`KL ASYMMETRY OK` + `PPL CROSSCHECK OK` + 熵下限 `H` 已算出并抄进 NOTES + NOTES 累计 ≥8 条。

Day 3 • 周三：makemore bigram——同一个分布，两种得到它的方式

今日总览（6.5h）：今天进入 makemore。bigram 模型只有一个 27×27 的对数概率表，**这件事有两种做法**：直接数频率（查表版，无梯度）、或用梯度下降去优化它（micrograd 版，就是昨天推的 `p-y`）。今天两条都要做，并且都要**收敛到你昨天算出的那个** `H`。这一步做扎实了，Day 4 换 MLP 时你才知道模型变复杂、loss 下界也会变（因为上下文变长了），而不是背一句「参数多就更准」。

任务 1：makemore 视频上半场（约 100 分钟）

1. 看 Karpathy「Neural Networks: Zero to Hero」系列第二讲 **「The spelled-out intro to language modeling: building makemore」**（YouTube 原链在系列播放列表里；B 站搜「makemore karpathy」有中文字幕搬运，非官方但够用）。
2. 今天看到他把 bigram 模型讲完、开始说「one-hot 向量之后我们要一个 MLP」为止；**按内容切点，别掐时间戳。**
3. 硬性看法：他写一行你写一行，落到 `week02/day3/follow_bigram.py`；每个新名词（分词、词表、上下文窗口、`multinomial` 采样、温度）出现时暂停 30 秒，在 NOTES 写一行自己的话。
4. 他的 `get_batch` 你会照抄——照抄可以，但要在旁边注释里写清 `xs` 和 `ys` 各自错开几位（这就是「用前 t 个预测第 t+1 个」）。

完成判据：`follow_bigram.py` 能跑出 loss 下降的打印；NOTES 新增 ≥4 条含「分词 / 词表 / 采样 / 温度」四个词的解释。

> 注：makemore 的 bigram 部分不需要 GPU，全程 CPU；他的视频时长比 micrograd 短，但**信息密度更高**（一次讲清 tokenization、batch、采样、loss 与 PPL 的关系）。如果昨晚的 CE 推导你今天还记得，看视频时会有"他在念我昨天写的稿子"的感觉——抓住这个感觉，把它写进 NOTES，那是本周最好的正反馈。

任务 2：数据体检与预处理（约 50 分钟）

在 `week02/day3/data_check.py` 里打印：行数（= 名字数）、总字符对数 `N`、每个名字长度的均值/最大、出现频次 top10 与 bottom5 的字符、以每个字符为「前文」时的下一字符分布熵 top3（用昨天 `bigram_entropy.py` 里的 `cond`，逐行算 `-Σ p log p`）。

完成判据：六项全部打印出来，且 `N` 与昨天算的 `N` 一致；NOTES 里挑一个「下一字符熵最低」的前缀并解释为什么它低（通常是某个几乎固定的结尾，比如 `ng` → `.` 之类的强约束；以你的实测为准，别抄别人的结论）。

> 注：熵最低的那些前缀就是模型最好学的部分，熵最高的那些（通常是词首）决定 loss 的下限。**「数据里哪一部分不可约」**这个直觉，是 Week 7 做困惑度过滤、Week 9 判断 reward hacking「模型在背模板」时的底层能力。

任务 3：查表版 bigram——numpy 数频率，torch 采样（约 90 分钟）

新建 `week02/day3/bigram_table.py`，先自己写：

```
"""bigram 查表版:不做梯度,直接数频率。它是所有 bigram LM 的 MLE 解。"""
import numpy as np, torch

H_REF = 2.18        # TODO: 改成昨天 bigram_entropy.py 打印的那个 H,不许用我的数
path = "E:/study/posttrain/week02/data/names.txt"
names = [n.strip() for n in open(path) if n.strip()]
seqs = [["."] + list(n) + ["."] for n in names]        # 每个名字自成一条序列,不跨名字配对
chars = sorted(set(c for s in seqs for c in s))
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
V = len(chars)
cnt = np.full((V, V), 1e-8, dtype=np.float64)          # 微小平滑,避免 0 概率
for s in seqs:
    for a, b in zip(s[:-1], s[1:]):
        cnt[stoi[a], stoi[b]] += 1
probs = cnt / cnt.sum(axis=1, keepdims=True)            # 每行归一化 = 条件分布
log_probs = torch.tensor(np.log(probs))

pairs = [(stoi[a], stoi[b]) for s in seqs for a, b in zip(s[:-1], s[1:])]
ce = float(-torch.stack([log_probs[a, b] for a, b in pairs]).mean())
print(f"查表版平均 CE = {ce:.6f}   昨天的 H = {H_REF:.6f}   diff = {abs(ce-H_REF):.2e}")
assert abs(ce - H_REF) < 1e-3, "两个脚本的序列约定不一致,先对齐再往下做"

g = torch.Generator().manual_seed(1337)                 # seed 写死:采样结果要能复现给面试官看
out = []
for _ in range(30):
    i = stoi["."]; s = ""
    while True:
        j = torch.multinomial(torch.exp(log_probs[i]), 1, generator=g).item()
        if j == stoi["."]:
            break                                       # TODO: 这行以前先自己想清楚何时该停
        s += itos[j]; i = j
        if len(s) > 20:
            break
    out.append(s)
print(out[:15])
print("TABLE BIGRAM OK")
```

说明与要求：`break` / `while True` 那段故意给成半成品——**「起始符号是 `.`、结束符号也是 `.`」这个状态机要你先在纸上画清楚再改对**，直接跑会出死循环或空串，这是设计给你的第一个采样 bug。

完成判据：查表版平均 CE 与你昨天的 `H` **差 ≤1e-3**（脚本里已 assert，两个独立实现对上才说明序列约定一致）；采样 30 个名字里 **≥20 个**长度在 2-14 且只含 `a-z`；打印 `TABLE BIGRAM OK`。

> 注：查表版算出的平均 CE 是**训练集上的**，天然偏低（它把频率当概率，等于把噪声当知识）。这就是「训练 loss 很低但采样出来不像话」的第一个真实案例——Week 6 做 SFT 时你会在 train/eval 上再撞一次，那时你该想起今天。

任务 4：梯度版 bigram——用 micrograd 优化那张 27×27 表 + 采样对比（约 135 分钟）

新建 `week02/day3/bigram_sgd.py`：用 Week 1 的 `Value` 引擎（`sys.path.insert(0, "E:/study/posttrain/day2")`），参数是 27×27 的 `Value` 表（729 个参数），前向取 `logits = W[prev_char]` 这一行，loss 用**昨天手推的形式**自己写（`softmax + CE`，含 `/batch_size`）。要求：

1. `batch_size=32, lr=1.0`，跑 **500 轮**（这就是 Karpathy 视频里的量级，纯 Python 下预期**几分钟到十几分钟**，比 Day 4 的 MLP 快得多）；每 50 轮打印 loss。
2. 记录 loss 曲线到 `week02/day3/bigram_loss.png`，并算「最终 loss 与 `H` 的差」。
3. 加一个开关：`lr = 100`（默认 1.0），记录它炸不炸——这个 100 倍 lr 是 Day 5 曲线图的热身。

把 `lr=100` 那一次的最终 loss 或 `nan` 记进 NOTES（这是 Day 5 扫参的热身）。

> 注：如果你发现梯度版怎么调都停在 `H` 上方 0.3 以上，先查两件事：① 你有没有做 batch 归一化（`/N`）② lr 是否太小、500 轮是否真的跑满。别急着上 BatchNorm——这不是它的活儿，bigram 是个凸问题（softmax 表 + CE 在 logits 上凸），**优化不到就是 lr / 步数 / 归一化**。这个"凸问题不收敛 = 优化配置错，不是模型能力问题"的判断，是 Week 5 排查训练不稳定时的第一块拼图。

同一 seed（1337）下把查表版和梯度版各采 30 个名字并排贴进 NOTES，回答两个问题：哪个更像名字？loss 更低的那个采样一定更好吗？

完成判据（本任务含采样对比）：最终 loss ≤ `H` +0.15 且打印 `SGD BIGRAM OK`；NOTES 里有两组各 30 个样例 + 一段 ≥3 行的结论（含「训练 loss 是平均意义上的，采样看的是尾部」这个意思）。

任务 5：收工笔记（约 15 分钟）

今天最痛的卡点 1 个 + 明天 MLP 版的预期（提示：上下文从 1 个字符变成 3 个，loss **应该**比今天的 `H` 低，为什么）。

完成判据：NOTES 有「查表版 vs 梯度版」差异的一句话（loss、采样、耗时三项里至少两项），且明天预期的那句你写了具体数字（例如「我预期 ctx=3 能到 2.1x，因为…」）。

今日验收（6.5h）：`TABLE BIGRAM OK`（CE 与 `H` 差 ≤1e-3）+ `SGD BIGRAM OK`（最终 loss ≤ H+0.15）+ 两组 30 个采样样例 + NOTES 累计 ≥12 条。

Day 4 • 周四：MLP 语言模型——上下文、one-hot 与嵌入

今日总览（6.5h）：把 bigram 换成「看前 3 个字符预测第 4 个」的 MLP：one-hot 上下文 → 隐藏层（tanh）→ 27 类。今天你会亲眼看到两件事：**加上下文确实降 loss（因为下界变了，不是模型变魔法）**，以及 **one-hot 输入等价于「查表取对应行的权重」，这就是 embedding 的前身**。晚上做一次窗口大小消融（1/3/6），这张小表直接进 Week 5 博客的素材库。

任务 1：视频下半场（约 75 分钟）

从昨天停的地方看到「我们要把这套东西变成 GPT」之前。重点跟抄 `get_batch` 的 one-hot 版本和 `MLP` 的 `uniform_` 初始化（他会解释为什么权重初始化不能是 0，也不该是标准正态）。

完成判据：`week02/day4/follow_mlp.py` 存在且能跑出 loss 下降；NOTES 写一条「one-hot 乘权重矩阵 = 取那一行」（用你自己的话，含维度论证）。

任务 2：搭 `nn.py`（约 80 分钟）

从 Week 1 的 `day2/nn.py` 拷到 `week02/day4/nn.py` 并扩充三样：

1. `Layer`：权重从 `torch.zeros` / `uniform_(-0.5, 0.5)` 初始化，并支持 `bias`；
2. `Tanh` 逐元素；
3. 一个 `LogisticNeuron/softmax + CE` 的层，用于输出 27 类，loss 就用 Day 1 推的 `p - y`（自己写 `_backward`，别偷懒用 `Value.log` 手搓 CE——先写对的，再对比）。

完成判据：`python -c` 能 import 且 81→100→27 的三层前向打印出的概率行和为 1（误差 <1e-6）。

> 注：在 micrograd 里手写 CE 的 `_backward`，你会立刻明白「为什么框架要内置 `F.cross_entropy`」——不是数学难，是**逐标量记账太慢且容易漏 `+=`**。手写一次（今天），之后一律用 torch（Week 3 起）。这是本周关于「手撕 vs 调包」的唯一一次强制手撕，也是面试手撕题的原题。

任务 3：训练 MLP 语言模型（约 110 分钟）

`week02/day4/train_mlp.py`：上下文窗口 `ctx = 3`，输入维 `3*V = 81`，隐藏层先用 **30**（tanh，别一上来就 100——参数 81×30 与 81×100 在纯 Python 下差 3 倍多的时间），输出 27；batch 16，训练 **先 100 轮看趋势**，再决定跑到 300-500 轮（micrograd 逐标量算，整个训练预期在 **10-40 分钟量级**，屏幕很久不动是正常的；等不起就降到 100 轮 + 隐藏 20，并在 NOTES 记一句「我为了省时间牺牲了什么」，别改成 numpy——今天的目的是"同一个 loss 家族"，不是速度）。

要求打印：每 50 轮的 loss、以及最后 20 个采样名字。

完成判据：MLP 的最终 loss **比 Day 3 的 bigram 经验熵 `H` 低 ≥0.03**（上下文更长 → 可预测性更高 → 下界本身下降，这是"为什么多轮上下文的 LM 更强"的第一个实测证据；若没低，先查 `ctx` 是不是真的传进去了）；采样 20 个里 **≥15 个**是「像名字」的（长度 2-14、无重复字母堆叠如 `aaaa`）；打印 `MLP LM OK`。

> 注：一个必踩的坑——one-hot 上下文若把「`.` 填充」和「真实字符」混在同一维度顺序里，不同名字会共享错误的位置。写 `for t in range(ctx): ...` 时把注释写成 `# t=0 是最远的前文,t=ctx-1 是紧邻的前文`，并与 `get_batch` 的切片严格对应。这类 bug 的特征是：**loss 能降到差不多，但采样的名字开头总有一两个怪字符**，因为它学到的是错位的关系。

任务 4：窗口大小消融 1 / 3 / 6 与采样质量对照（约 115 分钟）

1. 同一套代码、同一 seed，只改 `ctx ∈ {1, 3, 6}`，各跑相同轮数，记 `loss`、`PPL=exp(loss)`、采样 10 个名字的主观"像名率"（人工数）。
2. 把 bigram（查表）、bigram（梯度）、MLP ctx=1/3/6 的采样样例并排贴进 `week02/day4/samples.md`。

3. NOTES 写 ≥3 条：为什么 MLP 比 bigram 好；为什么"好"必须同时看 loss 和样本来判断；温度/top-k 如果加进来会怎样（今天不实现，只写预期，Week 5 会真做那张对照表）。

完成判据：`week02/day4/ctx_ablation.md` 一张 3 行表（顶层表格，不嵌进列表），三列 ctx / loss / PPL，且 **loss 随 ctx 单调不增**（ctx=1 那行应约等于 Day 3 的 bigram 结果，用它当自校验）；`samples.md` 里 ≥5 组样例（每组 ≥20 个名字）；NOTES 三条齐。

> 注：ctx=6 有时**采样质量不升反降**，因为参数（`ctx*27*hidden`）随窗口线性变多而数据没变多。把这句话写进表下方，并明确「小模型上容量/数据/上下文的三角权衡」——这是 Week 7 数据配比、Week 12 「为什么不直接上更大模型」的同一道题。

任务 5：收工笔记（约 10 分钟）

NOTES 写一条今天真正的收获：「one-hot 输入乘权重 = 取权重矩阵的对应行，这就是 embedding 的前身」，并用 float64 验证：构造只在下标 `k` 处为 1 的 one-hot，比较 `onehot @ W` 与 `W[k]` 是否 `allclose`；再解释「ctx 个字符拼接 = 这几行相加，而每行的下标是 `t*V + k`」——这句写对，Week 3 的 `nn.Embedding` 就不需要重新学。

完成判据：NOTES 该条有具体数字或 `allclose=True` 的打印，不是只有结论句。

今日验收（6.5h）：`MLP LM OK`（loss 比 `H` 低 ≥0.03）+ ctx 消融表 3 行 + `samples.md`。

Day 5 • 周五：优化器与学习率——SGD / 动量 / Adam / AdamW，手写一遍再扫参

今日总览（6.5h）：闭合 Week 1 自测第 5 题（Adam 两个动量式 + 偏差修正），并把「lr 是超参里最贵的那个」变成你自己的体感：16 条曲线的对比图，其中必须有一条炸给你看。今天的产出图，Week 5 调 nanoGPT、Week 6 定 LoRA lr 时都会被你翻出来。

任务 1：四个优化器的 numpy 版 + 偏差修正演示（约 80 分钟）

新建 `week02/day5/optimizers.py`，实现四个函数，签名统一 `update(params, grads, state, lr) -> None`（原地改 `params`）：

1. `sgd`：`p -= lr * g`
2. `momentum_sgd`：`m = mu*m + g; p -= lr*m`（说明与 `m = 0.9m + 0.1g` 的写法只差一个常数）
3. `adam`：`m = b1*m+(1-b1)g; v = b2*v+(1-b2)g²; m̂ = m/(1-b1^t); v̂ = v/(1-b2^t); p -= lr*(m̂/(√v̂+eps))`
4. `adamw`：在 adam 基础上 `p -= lr*(... + wd*p)`，**wd 不经过 m/v**

再写一个 8 行的偏差修正演示（`week02/day5/bias_correction.py`）：`m=v=0` 起步，喂 `g=1.0`，打印一步后的 `m、v、m/(1-b1)、v/(1-b2)`。

完成判据：`bias_correction.py` 打印出未修正 `m=0.1, v=0.001`（b1=0.9, b2=0.999）、修正后 `m̂=1.0, v̂=1.0`，即**恰好等于 g 与 g²**；NOTES 写一句「不做偏差修正，首步等效 lr 被 √v̂ 放大/被 m̂ 缩小，等效步长约

为修正后的 0.1/0.0316 ≈ 3 倍量级（以你打印的数为准）」；四个 update 函数在同一个 1D 二次函数 `f(x)=x², x0=3` 上 100 步都收敛到 `|x|<1e-3`（打印各自最终 x）。

> 注：Adam 的第一步在**不修正**时步长其实是 `0.1g/(0.0316|g|) ≈ 3.2·lr`，也就是"偏大"，而很多人背的是"估计偏小所以要修正"——注意区分：`m̂` 偏小（÷(1−β1) 是放大），`v̂` 也偏小（开根后放大分母 → 步长变小），净效应取决于 `g` 的一致性。今天你手上有打印出来的数，**面试时别说背来的结论，说你自己算的**。这种「用实测推翻自己背的句子」的能力，比多推一遍公式值钱。

任务 2：把优化器接到 Week 1 的 micrograd 网络上（约 80 分钟）

`week02/day5/train_moons_optim.py`：复用 `day2/engine.py` + `day2/nn.py` 与同一份 moons 数据（`make_moons(n_samples=100, noise=0.15, random_state=42)`，标签转 ±1，平方 hinge loss），**但把更新循环换成任务 1 的四个函数**（`params = [p.data for p in model.parameters()]` 与 grads 同步维护）。跑一个配置：`lr=0.05`，200 轮，打印 loss/acc。

完成判据：四个优化器各跑完并打印最终 acc，且**至少两个**达到 ≥90%；NOTES 记录四个的 rank（哪个最快到 90%）。

> 注：micrograd 是逐标量 Python 算的，100 点 × 337 参数 × 200 轮单次约 3-5 分钟；四个优化器跑满就是十几分钟，屏幕静默是正常的，别 Ctrl+C。**不要在这个版本上做扫参**——那是任务 3 用 torch 干的事，理由见下。

任务 3：torch 版等价实现 + 一致性验证（约 70 分钟）

`week02/day5/train_moons_torch.py`：同一数据、同一网络拓扑（2-16-16-1，tanh，平方 hinge），但用 `torch.optim.SGD/Adam/AdamW` 与手写循环各跑一遍 `lr=0.05`、200 轮。

完成判据：两版的**第 1、第 50、第 200 轮 loss 相对差 <1e-3**（浮点路径差异，不要求逐位相同）；打印 `OPTIM EQUIV OK`。

> 注：为什么允许 1e-3 而不是 1e-12？因为 torch 的 GPU/CPU kernel 融合、`foreach` 批处理与 micrograd 的逐标量顺序不同，累积轨迹必然分叉——**"两个正确的实现跑出不同结果"在随机优化里是常态**，判据要设成"轨迹级别一致"而不是"逐位一致"。这条认知到 Week 9 对 GRPO 的 reward 曲线复现时能救你一晚上。

任务 4：lr 扫参与曲线对比图 + 读图写结论（约 145 分钟）

`week02/day5/sweep.py`：4 优化器（SGD、momentum SGD、Adam、AdamW）× 4 个 lr（`[0.003, 0.01, 0.05, 1.0]`，SGD 与 Adam 的量程差别很大，**如果 1.0 没炸就加到 3.0/10.0，目标是图上必须有一条发散**），用任务 3 的 torch 版跑 300 轮，记录每轮 loss；`matplotlib.use("Agg")` 画出 16 条曲线（4 个子图，每个优化器一个，y 轴用 `symlog` 或截断到 1e3 才能看清发散的那条），保存 `week02/opt_sweep.png`。

再读图写三句话：① lr 太小/太大的症状分别是什么（慢 vs 震荡/发散，图上有证据）② momentum 为什么能把震荡抹平（等效低通滤波）③ Adam 为什么在**初期**快、在**后期**有时不如调好 lr 的 SGD+momentum（泛化差异，写"这是有争议的话题，我知道两派理由"比硬下结论更安全——面试追问时你有的说）。

完成判据：图存在且**恰好 16 条曲线**（图例数量核对）；至少 1 条曲线最终 loss > 1e3 或变 `nan`；4 个子图各有一条"最快下降"的 lr；`week02/day5/sweep_conclusions.md` 有上面那三句 + ≥5 行结论，**每句都指回图上的具体**

曲线（含「Adam 对 lr 尺度更不敏感，但只是看起来」这类被你数据支持或推翻的判断）。

> 注：这张图是本周面试可展示物之一，值得花时间把坐标轴、图例、标题弄干净。**别用** `plt.show()`，Windows/无显示环境下会挂住进程，看起来像死循环；统一 `savefig` 再关掉 figure。扫参用 torch CPU 版，全部 16 次跑完预期在**十几分钟到半小时量级**，超时就先减到 4×2 网格，别减到 4×1（两个 lr 点画不出"上限"这个概念）。

任务 5：收工笔记（约 15 分钟）

在 `sweep_conclusions.md` 末尾补一行「如果只能给 Week 5 的 M1 训练留一条 lr 建议，我留什么」，必须引用今天图上的具体 lr 值。

完成判据：该行存在且含一个具体 lr 数字；NOTES 新增 ≥2 条。

今日验收（6.5h）：`bias_correction.py` 数值符合预期 + `OPTIM EQUIV OK` + `opt_sweep.png`（16 条 + 1 条炸）+ NOTES 累计 ≥14 条。

Day 6 • 周六：收口——把 toy 换成莎士比亚、整理 repo、结算 gaps

今日总览（6h）：今天不学新概念，做三件事：① 把 bigram 从人名换成莎士比亚字符流（Week 5 M1 就是这个数据，**今天的 20 分钟是为周四那 12 个卡时省 3 小时**）；② 把这周代码整理进 `week02/makemore/` 并推上 GitHub；③ 结算 `week02-gaps.md`，把新暴露的洞映射到 Week 3/4/5。 optionally 开 30 分钟卡，看一眼同一份代码在 GPU 上跑成什么样。

任务 1：数据通用化——莎士比亚字符流（约 60 分钟）

1. 拿数据（两条路，任选一条走通就停）：
   - 首选 nanoGPT：`git clone https://github.com/karpathy/nanoGPT E:/study/posttrain/nanoGPT`，按仓库 `data/shakespeare_char/readme.md` 与 `prepare.py` 生成 `input.txt` 与 `meta.txt`（它需要联网下载原书纯文本；下不动就走第 2 条）。
   - 降级：gutenberg 的《The Complete Works of William Shakespeare》纯文本，自己用 Day 1 的 `CharVocab` 编解码；**两条都不通就继续用 names.txt**，本周结论与数据无关，Week 5 再补。
2. 把 `bigram_table.py` / `bigram_sgd.py` 的 `path` 与 `seqs` 抽成 `load_dataset(name)`，字符级切分改成「每 100 字符一段、段间加 `.`」这类你自己能跑通的约定（**莎士比亚是连续文本，不是行列表**，别照抄人名逻辑）。
3. **必做**：NOTES 写一句「人名模型对 names.txt 是严重过拟合的小 toy」，并附真实证据（同一模型在 names.txt 与一小段 unseen 文本上的 CE/PPL 各多少，**跑出来的，不许凭记忆**）。

完成判据：`load_dataset` 支持两种数据源都能训练；NOTES 有「同一模型在见过的数据 vs 没见过的数据上的 CE/PPL 各是多少」这一对实测数（**不许写记忆里的数，跑出来的**）；打印 `DATA OK`。

> 注：这是本周第一次真正体会「train loss ≠ 模型好」。你现在手上的 27×27 表能记住 3.2 万个名字的所有相邻字符对，而它拿到一段没见过的英文时 PPL 会明显更高。Week 7 的困惑度过滤、Week 12 的"eval 集不能进训练集"，根都在这一条。

任务 2：可选开卡 30 分钟（约 45 分钟含开机/关机）

1. 开 AutoDL 一台 4090（按量计费），镜像选 PyTorch 2.x + Python 3.10 + CUDA 12.1；**开卡前**先把要跑的脚本和启动命令写进 `week02/day6/run_on_gpu.sh`，跑完立刻在控制台点**关机**（不是销毁）。
2. 只跑一件事：把 `bigram_sgd.py` 用 torch 的矩阵乘/ `index_add` 改写（或直接跑 Day 5 的 sweep 缩到 2×2），对比同一 seed 下 CPU 与 GPU 的墙钟时间与 loss 一致性（相对差 <1e-3）。顺手把三件运维事实各写一行进 NOTES：控制台「已关机」长什么样、 `/root/autodl-tmp` 与 `/root` 的区别（关机≠销毁， `/root` 可能随镜像重置丢）、 `source /etc/network_turbo` 对 github/pypi 生效。

完成判据：NOTES 里有 CPU vs GPU 的墙钟对比（两个数 + 你的解释：为什么这么小的模型 GPU 不一定更快——Python 端 kernel 启动开销主导）+ 三件运维事实。**没抢到卡不算失败**，改用任意有货的卡或直接跳过并把「补做 GPU 对比」记进 gaps（Week 4 反正要上卡）。

> 注：这 30 分钟是体验，不是任务。本周卡时上限 0.5h、目标花费 ≈¥0-2，超了就是没走。

任务 3：repo 收口与 push（约 120 分钟）

1. 建 `E:/study/posttrain/week02/makemore/`，把本周文件收进去： `data/names.txt`（来源：随 makemore 仓库分发的公开数据，README 里注明）、 `vocab.py`、 `ce_check.py`、 `kl_asym.py`、 `ppl_check.py`、 `bigram_entropy.py`、 `bigram_table.py`、 `bigram_sgd.py`、 `train_mlp.py`、 `optimizers.py`、 `sweep.py`、 `opt_sweep.png`、 `samples.md`、 `README.md`。
2. README 四段：这是什么 / 怎么跑（一条命令一条）/ 关键数字（ `V`、 `N`、 `H`、bigram 收敛 loss、MLP 最终 loss、PPL vs 基线 27）/ **哪些是跟着 Karpathy 敲的、哪些是你自己写的**（面试诚信声明，必须写实）。
3. 拷一份进 `llm-posttraining-lab/notes/week02-makemore/`， `git add` 相关文件后 push。**不要**把 `names.txt` 之外的原始大文件提交（ `git status` 看一眼体积）。

完成判据： `cd week02/makemore && python ce_check.py && python ppl_check.py && python kl_asym.py` 三连全 OK（在新目录里独立可跑，不依赖 `dayX/` 的路径）；GitHub 链接可访问且 README 渲染正常；NOTES 顶部有该链接。

> 注：三连能在**新位置**跑通，才是「可复现」；只是在你 day1-day5 的散文件里能跑，那不叫产出，叫一堆脚本。改 import 与路径时优先用相对路径 + `pathlib`，别写死绝对路径（Week 5 要在服务器上跑同一份代码）。

任务 4：gaps 记账、新洞映射与素材归档（约 100 分钟）

1. 先把 Week 1 的 B 类三条结算： `week1-gaps.md` 里 softmax+CE/p−y、KL 与熵、Adam/偏差修正/学习率三条各标 `[x]` 并写一句「Week 2 哪个任务闭合、复测结果」。第 4 题（Transformer Block）**保持未勾**，并在文字里写明「已确认推给 Week 3 Day 1-3」。
2. 新建 `week02/week02-gaps.md`，格式沿用「错了什么 → 反映哪个空洞 → 补在哪天 → 用什么动作补」，**至少 6 条**，且新洞必须落到下面这张映射表的具体格子里：

本周暴露的洞（示例，按你实际情况写）	归到	为什么那天会自然补上
多头/带 batch 的 attention、Block 整体结构、位置编码怎么进网络	Week 3 Day 1-2	本周把 (B,T,C) 与 PPL 都摸清了，下周第一件事就是把它升级成多头
采样/温度/top-k 只在 toy 上试过，没有和真实 GPT 采样对照	Week 5	M1 交付里有 temperature/top-k 对照小表
优化器状态的显存代价、lr 调度（warmup+cosine）在真训练里怎么配	Week 4 Day 3	那周手写 AdamW 与调度器并和 `torch.optim.AdamW` 逐迭代对拍
BatchNorm/Dropout/嵌入的高级用法（视频后半段没跟）	Week 4 Day 2 + Week 5	训练循环模板里 `model.train()/eval()` 就是 dropout 的行为开关
梯度累积、混合精度、checkpoint 断点续训（本周完全没碰）	Week 4 Day 2/Day 4	那周的主线就是「会跑训练」
长上下文/真实分词（BPE）而非字符级	Week 5 + Week 6	nanoGPT 的 BPE 路径与 Qwen 的 tokenizer 会逼你面对

4. 过一遍 `NOTES.md`，确认本周新增 **≥15 条**（Day 1 起算），不够就从今天的 push 过程里补真实的坑（不许写口号，必须「现象 → 根因 → 修法」）；把 `opt_sweep.png`、`samples.md`、ctx 消融表拷进 `llm-posttraining-lab/blogs/drafts/week02-materials/`，并写一行「这些素材进 Week 5《我从零训了一个 GPT》的哪一节」。**本周不发布博客**，第一篇排期在 Week 5。简历 v0 里加一行占位（Phase 1 完成度 + makemore 链接），别等 Week 13 再回忆。

完成判据：`week1-gaps.md` 三条 B 类标 `[x]` 且有复测结论、第 4 题明确写着「Week 3」；`week02-gaps.md` ≥6 条且**每条**都有「Week 几 + 哪个任务」（出现「以后补」「有空再看」视为未完成）；NOTES 本周新增 ≥15 条（数一遍，把数字写进 review.md）；`blogs/drafts/week02-materials/` 三样文件 + 一行归属说明。

任务 5：周复盘 review.md（约 35 分钟）

`week02/review.md` 五块：① 硬产出清单（链接/路径）② 上周 B 类三条现在的掌握度（自评 + 复测依据）③ 本周真实时间花销（对照 42h，按天记）④ 最有价值的一件事 ⑤ 下周最想补的一个洞。

完成判据：五块齐全，且第 ③ 块是**按天记的真实数**（七行，与总览表逐项对比，写清哪一天超了几小时、超在哪）；第 ② 块不许出现"基本掌握"这类没有依据的词。

今日验收（6h）：`week02/makemore/` 三连对拍全 OK 且已 push + `week02-gaps.md` ≥6 条有归属 + NOTES 本周新增 ≥15 条 + review.md 五块齐。

Day 7 • 周日：缓冲补漏 + 闭卷复测 + 总验收 + Week 3 预习

今日总览（4.5h）：今天不排新内容。补漏最多 2 个洞，做闭卷复测，然后验收，最后 30 分钟预习下周。**之后彻底休息**——Week 5 的 M1 是硬里程碑，把电留在周四。

任务 1：补漏（约 110 分钟）

打开 `week02-gaps.md` 和 `review.md` 的第 ② 块，**只补 2 个洞**（贪多必失，Week 1 的 A 类遗留优先）。补的方式必须是"重做那道题/那个脚本"，不是重读笔记。

完成判据：这两条在 gaps 里标 `[x]`，并各写一句「怎么补的 + 复测数值」。

任务 2：闭卷复测 3 道（约 40 分钟，别跳）

合上所有窗口，拿本周 Day 1/Day 2 的原题重做（写在本子上或 md，**不许看 NOTES、不许看代码**）：

1. 写出 `softmax + 交叉熵` 对 logits 的梯度（含 batch 归一化因子），并说明实现时为什么把两者合成一个算子、`F.cross_entropy` 的入参是什么。
2. 给出熵、交叉熵、KL 三个定义与关系式，说明为什么「CE 的最小值有下限、这个下限是什么」，以及为什么训练用 CE 而不是直接最小化 KL。
3. 写出 Adam 的两个动量更新式、偏差修正与参数更新式；再写 AdamW 与 Adam+L2 的区别（一行）。

计时 40 分钟，到点停笔。

完成判据：三题都写了（空着才算失败）；**合格线 = 第 1、3 题全对**（第 2 题允许 `H(p,q)=H(p)+KL` 的推导有笔误，但"下限就是数据分布的熵"这句必须自己写出来）。结果记进 `review.md` 第 ② 块（对/半对/错）。

> 注：这是整个计划里唯一真正检验「补上了没有」的环节。`[x]` 不算数——三天后依然会掉坑，这是学习曲线的基本事实，不是你的问题。三题里哪题写不出，就地写进 Week 3 的 Day 1 前 45 分钟（第 3 题写不出 → Week 3 用不到，但要挪到 Week 4 Day 3 的手写 AdamW 之前重做）。

任务 3：对答案与订正（约 55 分钟）

对照本周 Day 1 推稿与 Day 5 任务 1 的实现逐句订正，错的用红笔（或 `**加粗**`）标在复测原文上，不要重抄一遍干净的——**保留错误的痕迹是 Week 14「失败案例复盘」的原材料**。

完成判据：复测卷上有订正痕迹；至少 1 条订正写进 `week02-gaps.md` 并给归属周。

任务 4：总验收（约 35 分钟）

逐项勾选（结果抄进 `review.md` 末尾）：

#	验收项	判据
1	概率：MLE/MAP → CE	Day 1 推稿两式齐全，NOTES 有「weight decay = 高斯先验」；MAP 一行有 `1/(2σ²)` 对应关系
2	softmax+CE 梯度	`ce_check.py` 打印 `CE GRAD CHECK OK`（diff 1e-9~1e-7 量级）+ 手稿有 `p − onehot(y)`
3	信息论一手	`kl_asym.py` 打印 `KL ASYMMETRY OK`（一侧 `inf`、一侧 `ln 10 = 2.302585`，脚本内 assert 通过）；`H(p,q)=H(p)+KL` 数值验证通过
4	PPL 对拍	`ppl_check.py` 打印 `PPL CROSSCHECK OK`（CE diff <1e-12）+ 均匀基线 `PPL ≈ V = 27` 记进 NOTES

#	验收项	判据
5	bigram 双实现	查表版 CE 与自算经验熵差 ≤1e-3；梯度版 500 轮后 ≤ `H` +0.15 且较 `ln 27` 至少降 0.8；两组 30 个采样样例在档
6	MLP LM + 消融	`MLP LM OK`（loss 比 `H` 低 ≥0.03）+ ctx∈{1,3,6} 三行消融表且 loss 单调不增
7	优化器与 lr	偏差修正演示数值正确 + `OPTIM EQUIV OK` + `opt_sweep.png`（16 条曲线、≥1 条发散）+ NOTES 本周新增 ≥15 条 + `week02/makemore/` 已 push 且三连在新目录可跑

≥6 项通过 = 本周合格；不足 6 项，把缺的写进 Week 3 Day 1 的前 2 小时（但「2、4、7 三项不许全缺」——它们是 Week 4 手写 AdamW、Week 5 读 loss 曲线的直接前置）。

完成判据：验收表已抄进 `review.md` 末尾且 7 行都有状态标记（通过/未过 + 一行原因），不是只打勾。

任务 5：下周预习（约 30 分钟）

1. 打开 Karpathy 系列第三讲 **「Let's build GPT: from scratch, in code, spelled out」**，只看前 20 分钟，回答一件事：makemore 的 MLP 输入和 GPT 的输入差在哪（上下文窗口是"固定 3 个字符"vs"任意长序列 + 位置嵌入 + 因果掩码"）。
2. 把 Week 1 Day 5 那份 `attention.py`（只有 (T,C)、无 batch、无位置编码）打开，在文件顶部写三行 TODO：**升级成 (B,T,C) + 多头 + 位置编码**——这正是 Week 3 Day 1 的第一个任务。
3. `review.md` 末尾写一行本周的位置：micrograd（会算梯度）→ **makemore（会定义语言建模的损失、会看 loss/PPL）** → nanogpt（会搭 Block）→ PyTorch 训练系统（会跑训练）→ **之后才是后训练**：SFT/LoRA、DPO/RLHF、数据与评测。

完成判据：NOTES 里有「makemore 的固定 3 字符窗口 vs GPT 的变长序列 + 位置嵌入」这一句对照；`day5/attention.py`（Week 1 那份，在 Week 1 的 `day5/` 目录）顶部出现三行 TODO。

今日验收（4.5h）：验收表 ≥6 项打勾 + 复测第 1/3 题全对 + `attention.py` 顶部有三行 TODO。

三、常见问题与纪律

- **某天没完成怎么办？** 不熬夜追。按「天-任务号」写进 `week02-gaps.md`，Day 6 缓冲与 Day 7 就是为它们准备的。连续两天全崩就砍 Day 2 任务 5 与 Day 4 任务 4（保主线），**永远优先保 Day 1 梯度对拍、Day 2 PPL 对拍、Day 5 扫参图**——那三样是这周真正的能力增量。
- **卡住多久该求助？** 单点超 40 分钟：先写 NOTES（现象 + 你试过的三件事），再查官方文档/numpy/torch 的 issue；再 20 分钟无解就跳过并立 gaps 条目。**跳过不丢人，欠账不留名才可怕。**
- **对拍失败的排查顺序（本周通用）**：① dtype 是否统一 float64 ② 归约轴/`axis` 与 gather 的索引形状 ③ 有没有把 logits 当概率 ④ 序列/批次维是否搞反 ⑤ 是否两边都除了 N。**把 diff 打印出来**再 assert，只看 `AssertionError` 是在浪费时间。
- **钱**：本周目标 ¥0，上限 ¥2（Day 6 那 30 分钟）。"顺手多跑一会儿"一律记进 gaps 到 Week 4 补做——**P1 全程卡时上限 30h**，Week 5 的 M1 一个人就要吃 12h。

- **要不要二刷视频?** 不要整片重看，用做题代替。只有两种情况值得回去：① Day 7 复测第 1 或第 3 题写不出 ② Day 3 梯度版收敛不了、想确认 Karpathy 的序列约定。
- **AI 工具的使用边界**（本周放宽一档）：允许让它解释报错、讲清一个概念、review 你写好的文件；**不允许**让它生成 `ce_check.py` / `kl_asym.py` / `bigram_sgd.py` / `optimizers.py` 这四个文件的主体——它们是本周的肌肉记忆。README 里写清哪些是跟敲、哪些自写，这条诚信声明面试会被追问。